DOCUMENT

14

Appendix

Agent conventions and the feature briefs for future work

Agent conventions

Invoice approval demo app: a Python 3.10+ FastAPI service with a small static web UI (a Python port of the original ASP.NET Core demo). Specs are written with Spec Kit under `specs/`.

Layout

- `backend/src/invoice_app/domain`: business rules, no web or storage code. `ApprovalService` owns the invoice lifecycle (Draft, PendingApproval, Approved, Rejected). `approval_policy.py` holds the pure approval rules as small classes (`ApprovalRule`) combined by an `ApprovalPolicy`.
- `backend/src/invoice_app/api`: routers (`routers/`), sign-in (`auth/`), in-memory repository, `main.py` (composition root: `create_app` chooses and wires the implementations), `error_handling.py` (domain error to HTTP status) and the web UI under `wwwroot/`.
- `backend/tests`: pytest tests. One test module per module under test, named `test_<module>.py`. `fake_invoice_repository.py` is the shared fake.
- `e2e`: Playwright end-to-end tests (`tests/`) and page objects (`pages/`).
- `specs/`, `.specify/`: Spec Kit output (spec, plan, tasks). `docs/use-cases`: short briefs for features we may build.
- `.claude/`: project agents (`agents/`), skills (`skills/`), logging and validation hooks (`hooks/`). `logs/ai-interactions.jsonl`: the AI interaction log written by those hooks (27 fixed keys; `python scripts/summarize_log.py` validates it and writes `logs/ai-interactions-summary.md`).

Approval rules (the source of truth is `specs/001-invoice-approval-limits/spec.md`)

Invoice amount	Who may approve	Approvals needed
up to and including 50,000.00	Manager or Director	1
above 50,000.00, up to and including 500,000.00	Director	1
above 500,000.00	first: Manager or Director; second: **Director**	2 different users

- Limits are inclusive. Test one cent either side: 50,000.00 / 50,000.01 / 500,000.00 / 500,000.01. Money is `Decimal`, never `float`.
- The same user never counts as both approvers. Compare usernames after `strip().casefold()`.
- Only the roles `Manager` and `Director` can approve. `Admin` can reject and delete, never approve.
- A refused approval raises `ApprovalNotPermittedError` with a clear reason (HTTP 403 `{"error": ...}`) and changes **nothing**: all checks run before the first mutation.
- After the first of two approvals the invoice stays `PendingApproval`; `decided_by` and `decided_at` are set by the final approval only.

Layering and conventions

- Business rules go in `domain/`, never in routers or `main.py`. Routers translate HTTP to domain calls; domain errors map to status codes in `api/error_handling.py` (`ERROR_STATUS`): not found 404, wrong state 409, not permitted 403, bad input 400.
- Routers live in `api/routers/`, one per resource, prefix `/api/<resource>`. Request bodies are pydantic models declared in the router module.
- Every endpoint is behind sign-in (session middleware). Restrict actions with `Depends(require_role("..."))`: destructive actions are Admin only; approval endpoints allow Manager, Director and Admin, and the domain decides who may actually approve.
- Services are created in `create_app()` (`api/main.py`), the only place that picks concrete implementations. Storage contracts are small Protocols in `domain/repository.py` (`ApprovalStore`, `InvoiceReader`, `InvoiceWriter`, `InvoiceRepository`) with an in-memory implementation in `api/`.
- Every new rule or endpoint needs tests. Use `FakeInvoiceRepository` in unit tests. Any new repository implementation must pass `tests/test_repository_contract.py`.

Design rules (SOLID)

- **Single responsibility:** one reason to change per class or module. A rule class checks one rule; `ApprovalService` coordinates the lifecycle; routers translate HTTP; `error_handling.py` maps failures.
- **Open/closed:** to add an approval rule, write a new class with `refusal(attempt)` and list it in `standard_policy()`. Do not edit existing rule classes or `ApprovalPolicy`.
- **Liskov:** every implementation of a Protocol must behave the same on the contract (the contract tests run against the fake and the real repository).
- **Interface segregation:** depend on the smallest contract that covers your needs (the approval service takes an `ApprovalStore`, not the whole repository).
- **Dependency inversion:** services take their collaborators (policy, clock, store, user directory) as arguments; only `create_app()` builds concrete ones. Do not import a concrete class where a contract exists.
- Use clear names; no one-letter variables or boolean `flag` parameters.
- End-to-end tests use page objects and role-based locators (`getByRole`, `getByLabel`), never CSS or XPath selectors.

Commands

- Install: `pip install -e "backend[dev]"` (or `pip install fastapi uvicorn pytest httpx`)
- Run the app: `python -m uvicorn invoice_app.api.main:app --app-dir backend/src --port 5000` then open http://localhost:5000
- Run the tests: `python -m pytest backend/tests -q`
- Run mutation tests (WSL/Linux, mutmut 3 needs `fork()`): `wsl -d Ubuntu-22.04 -- bash scripts/run_mutation.sh <label>` → `docs/mutation/<label>.txt`; survivors: `scripts/show_survivors.sh`
- Rebuild the PDF documentation: `pip install -e "backend[docs]"` then `python scripts/build_docs.py` (sources: `docs/guides/*.md`, specs, `docs/pr-review.md`; output: `docs/pdf/`). Update the guides when rules or endpoints change.

- Run end-to-end tests: `cd e2e && npm install && npx playwright install chromium && npm test`

Review expectations

- Tests must be able to fail: each refusal test asserts the reason text **and** that the invoice is unchanged (deep copy compared by equality). A test that only checks an exception type does not count.
- Fakes must behave like the real thing (the fake repository records and stores updates), otherwise "forgot to save" passes.
- Judge unit tests by mutation score (`mutmut`), not by line coverage. Triage each survivor: add a test, or record why it is equivalent or cosmetic.
- Review the diff with the `pr-reviewer` agent. In the PR description every finding gets exactly one verdict: **fixed** (say how), **dismissed** (give a reason), or **deferred** (give a reason and where it is tracked). Format: `.claude/skills/review-findings`.
- Land features in small steps (single-approver limits, then the two-approver rule), each commit with a message that says what changed and why.

Demo accounts (not real credentials)

- `demo.manager` / `ChangeMe123!` (role Manager)
- `demo.director` , `demo.director2` / `ChangeMe123!` (role Director)
- `demo.admin` / `ChangeMe123!` (role Admin)

Seed invoices for trying the rules: `INV-2001` (75,000.00, needs a Director) and `INV-2002` (600,000.00, needs two approvers).

Never

- Commit real credentials, tokens or customer data.
- Add business logic to `main.py` , `error_handling.py` or routers.
- Edit an existing rule class to add a different rule; add a new class instead.
- Mutate an invoice before every approval check has passed.

Use case 1

Background

Finance operations approvers open every pending invoice one at a time. For a school district with 40 invoices in a batch, that is 40 separate approvals.

The idea

Approvers should be able to select several pending invoices and approve them together.

What the business asked for

- Select multiple pending invoices from one screen.
- See a summary before confirming: number of invoices, total amount, customers involved.
- Exclude individual invoices from the batch before confirming.
- Keep an audit trail: who approved what, and when, for every invoice in the batch.
- Invoices over $1,000 need a second approver before they are final.

Out of scope for the first release

- Email notifications (in-app only for now).

Open questions (good prompts for clarifying questions)

- What happens to the rest of the batch if one invoice fails?
- Can the same person be both approvers on a large invoice?
- Is there a limit on batch size?

Use case 2

Background

Today an invoice is approved or rejected as a whole. Two school districts asked for a way to approve less than the full amount when part of an invoice is disputed.

The change

An approver can approve an invoice for an amount lower than the invoiced amount, with a required note explaining why.

What the business asked for

- The approver enters the approved amount, which must be greater than zero and less than the invoiced amount.
- Both the invoiced amount and the approved amount are kept on the invoice.
- The invoice shows as partially approved, separate from fully approved.
- A note is required whenever the approved amount is lower than the invoiced amount.
- Existing approve and reject behaviour stays exactly as it is.

Where it touches the code

The existing approval flow: `ApprovalService`, `ApprovalsController`, and the Approvals tab in the web UI.